Spark 运行机制理解

第三题：详解 Spark TaskScheduler，贴 Spark Intellij 源码截图加分

一、TaskScheduler 概述

TaskScheduler 是 Spark 任务调度框架中的低层调度器，负责将 DAGScheduler 提交的 TaskSet 实际分发到集群中的 Executor 上执行。它是 Spark 作业从逻辑执行计划（DAG）转换为物理执行的关键桥梁。

```java
// TaskScheduler 接口定义（Spark 官方 API）
public interface TaskScheduler {
    void start();                                   // 启动调度器
    void submitTasks(TaskSet taskSet);              // 提交任务集
    void executorLost(String executorId, ExecutorLossReason reason);  // 处理 Executor 失联
    void setDAGScheduler(DAGScheduler dagScheduler); // 设置 DAGScheduler 引用
    String applicationId();                         // 获取应用 ID
    void killTaskAttempt(long taskId, boolean interruptThread, String reason); // 终止任务
}
```

TaskScheduler 目前唯一的实现类是 TaskSchedulerImpl，它维护了任务队列、Executor 资源信息，并支持 FIFO 和 FAIR 两种调度模式。

二、TaskScheduler 的核心职责

2.1 架构定位

```
DAGScheduler (Stage 级别调度)
        |
        | submitTasks(TaskSet)
        ▼
TaskScheduler (Task 级别调度)
        |
```

│ resourceOffer / launchTask

▼

SchedulerBackend (资源管理适配层)

│

│ 与 Executor 通信

▼

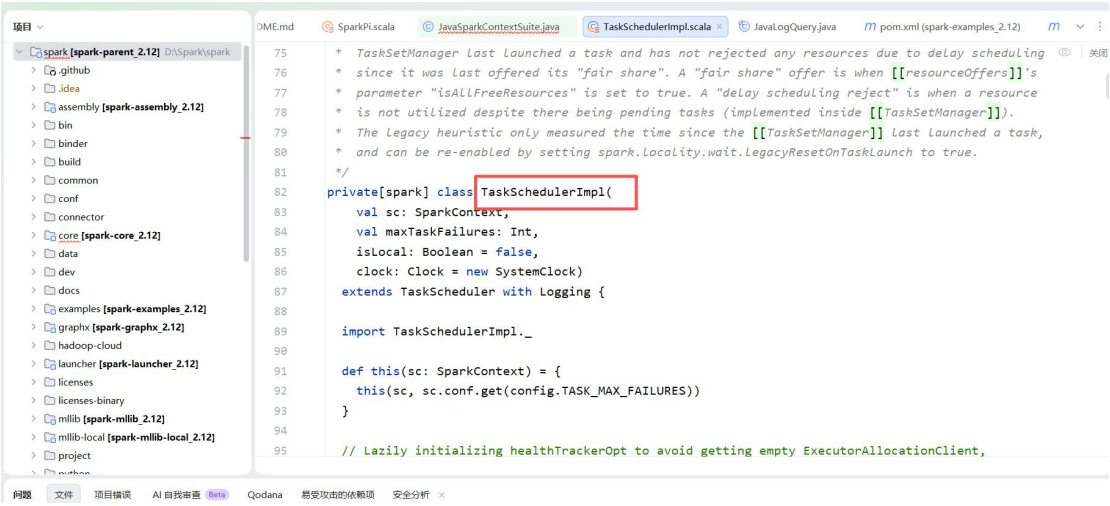
Executor (任务执行)

2.2 核心职责详解

职责	说明	关键方法
TaskSet 接收与缓冲	接收 DAGScheduler 提交的 TaskSet，创建 TaskSetManager 进行管理	`submitTasks()`
资源分配与调度	接收 SchedulerBackend 上报的资源，按调度策略为任务分配 Executor	`resourceOffer()`
任务生命周期管理	跟踪任务执行状态（成功/失败/重试），处理任务完成事件	`statusUpdate()`
故障处理	处理 Executor 失联、任务失败等情况，决定是否重试	`executorLost()`, `handleFailedTask()`
数据本地性优化	根据任务偏好位置（Process/Node/Rack）分配资源	`TaskSetManager.resourceOffer()`

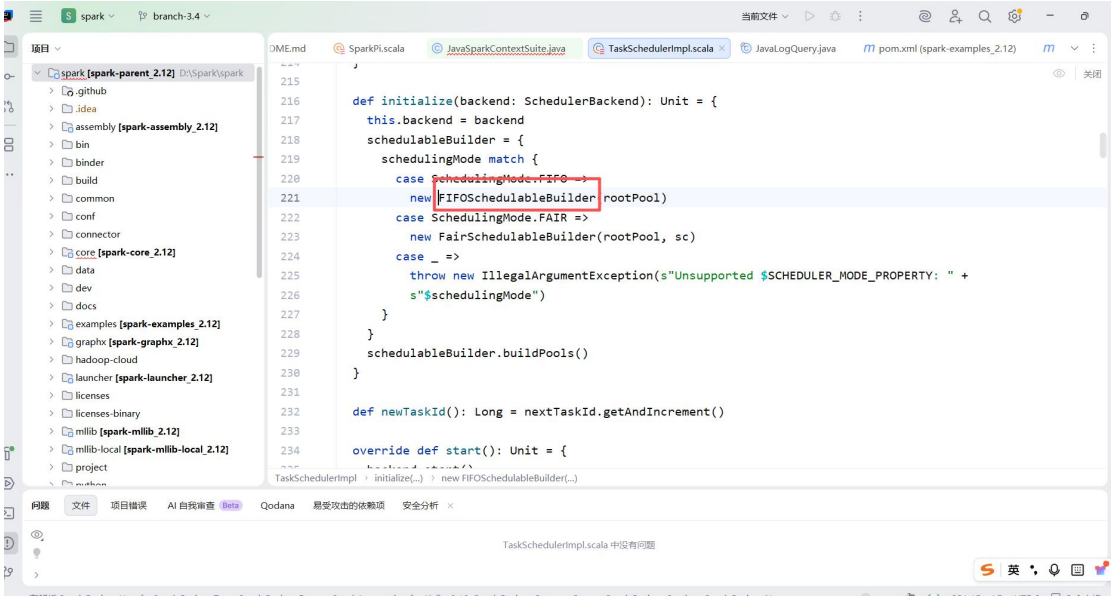
在 idea 中 spark 源码中查找 2.1 查找 SparkContext.createTaskScheduler 方法：

查找 TaskSchedulerImpl 类



查找调度模式相关类

查找 FIFOSchedulableBuilder：



查找 FairSchedulableBuilder：

```scala
def initialize(backend: SchedulerBackend): Unit = {
  this.backend = backend
  schedulableBuilder = {
    schedulingMode match {
      case SchedulingMode.FIFO =>
        new FIFOSchedulableBuilder(rootPool)
      case SchedulingMode.FAIR =>
        new FairSchedulableBuilder(rootPool, sc)
      case _ =>
        throw new IllegalArgumentException(s"Unsupported $SCHEDULER_MODE_PROPERTY: " +
        s"$schedulingMode")
    }
  }
  schedulableBuilder.buildPools()
}

def newTaskId(): Long = nextTaskId.getAndIncrement()
```

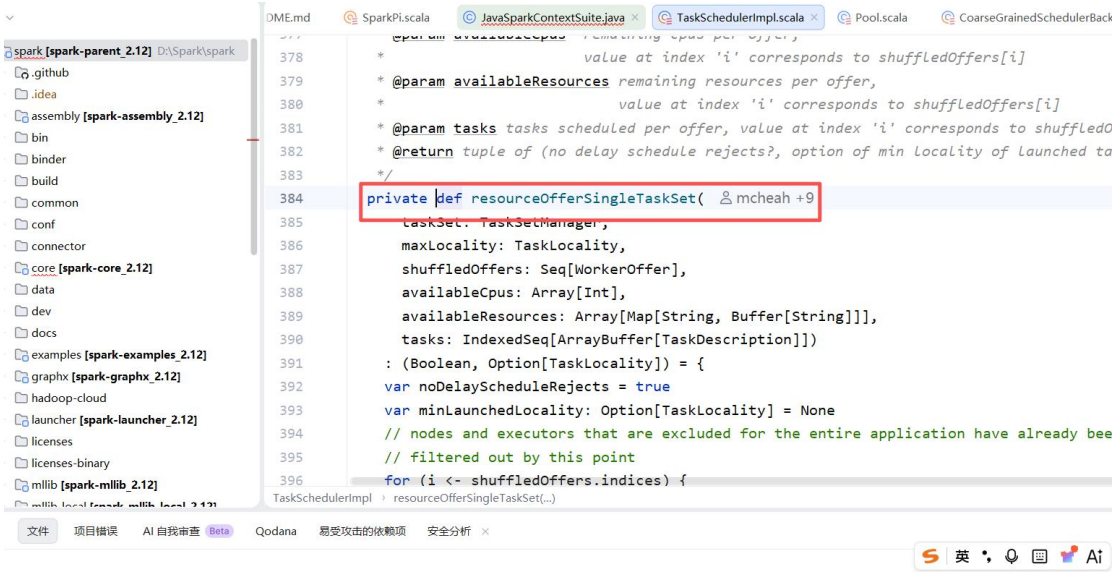
查找 Pool 类：

```scala
private[spark] class Pool(
    val poolName: String,
    val schedulingMode: SchedulingMode,
    initMinShare: Int,
    initWeight: Int)
  extends Schedulable with Logging {

  val schedulableQueue = new ConcurrentLinkedQueue[Schedulable]
  val schedulableNameToSchedulable = new ConcurrentHashMap[String, Schedulable]
  val weight = initWeight
  val minShare = initMinShare
  var runningTasks = 0
  val priority = 0

  // A pool's stage id is used to break the tie in scheduling.
  var stageId = -1
  val name = poolName
  var parent: Pool = null
```

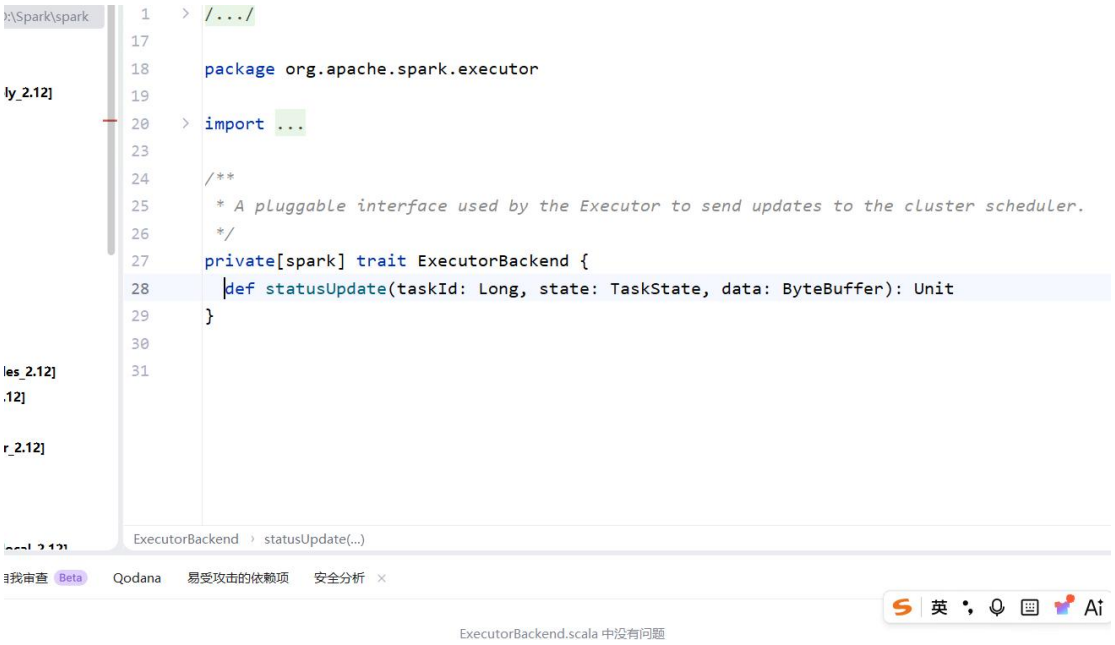
查找 CoarseGrainedSchedulerBackend

```scala
 * executors whenever a task is done and asking the scheduler to launch a new executor for
 * each new task. Executors may be launched in a variety of ways, such as Mesos tasks for the
 * coarse-grained Mesos mode or standalone processes for Spark's standalone deploy mode
 * (spark.deploy.*).
 */
private[spark]
class CoarseGrainedSchedulerBackend(scheduler: TaskSchedulerImpl, val rpcEnv: RpcEnv)
  extends ExecutorAllocationClient with SchedulerBackend with Logging {

  // Use an atomic variable to track total number of cores in the cluster for simplicity and speed
  protected val totalCoreCount = new AtomicInteger(0)
  // Total number of executors that are currently registered
  protected val totalRegisteredExecutors = new AtomicInteger(0)
  protected val conf = scheduler.sc.conf
  private val maxRpcMessageSize = RpcUtils.maxMessageSizeBytes(conf)
  private val defaultAskTimeout = RpcUtils.askRpcTimeout(conf)
  // Submit tasks only after (registered resources / total expected resources)
  // is equal to at least this value, that is double between 0 and 1.
  private val _minRegisteredRatio =
```

查找 TaskSchedulerImpl.resourceOffers



查找 statusUpdate 方法



五、TaskScheduler 与 DAGScheduler 的协作

text

```
    DAGScheduler                          TaskScheduler     |
  |     | submitMissingTasks(stage)            |     | ├── 创建 TaskSet                    |
  | └── taskScheduler.submitTasks(taskSet)  |     | ──────────────────────────────────→ |
```

```
  │                                    ├── 创建 TaskSetManager     │
        ├── 加入调度队列      │                                    └── backend.reviveOffers()     │
                                        │     │                                          ▼     │
                                    resourceOffer()     │                                    ├
── 分配资源      │                                    └── launchTask()     │
                    │     │ taskStarted(task, info) ◂ ────────────────────│     │◂ ────────────────
────────────────────────────────│     │ ├── 更新 Stage 进度                          │     │ └── 如果 St
age 完成，提交下一个 Stage          │     │                                          │     │ handleTas
kCompletion(event) ◂ ────────────────│     │◂ ──────────────────────────────────────────────│     │
 ├── 记录任务结果                          │     │ ├── 更新 MapOutputTracker                    │
│  └── 如果任务失败，决定重试或标记 Stage 失败      │
```

六、性能调优建议

6.1 并行度设置

```
// 全局并行度配置spark.default.parallelism = 200  // 建议为集群总核心数的 2-3 倍spark.sql.shuffle.p
artitions = 200  // Spark SQL Shuffle 并行度
```

6.2 调度模式选择

场景	推荐模式	原因
单一用户批处理作业	FIFO	简单高效，作业按顺序执行
多用户并发作业	FAIR	支持资源池隔离，保证公平性
交互式查询与批处理混合	FAIR	避免长任务阻塞短任务

6.3 数据本地性调优

```
// 本地性等待时间配置spark.locality.wait = 3000ms  // 默认 3 秒spark.locality.wait.process = 3000m
sspark.locality.wait.node = 3000msspark.locality.wait.rack = 3000ms
```

七、总结

TaskScheduler 是 Spark 作业物理执行的核心组件，它：

接收 DAGScheduler 的 TaskSet，创建 TaskSetManager 进行精细化管理
通过 SchedulerBackend 感知集群资源，按调度策略（FIFO/FAIR）分配任务
实现数据本地性调度，优先将任务分配到数据所在节点
处理任务生命周期，包括成功、失败、重试、Executor 失联等场景

理解 TaskScheduler 的源码对于 Spark 性能调优和问题排查至关重要，尤其是当遇到资源分配不均、任务倾斜、调度延迟等问题时，深入理解其机制是解决问题的关键。